电子产品生产过程中的抽样和决策优化

摘　要

针对生产过程中的决策问题，本文从抽样检测方法入手，建立了序贯检验—假设检验相结合的抽样方法，大幅降低了抽样次数，减低了抽样成本。并利用蒙特卡洛模拟和遗传算法，面临复杂的多阶段生产过程，从诸多决策方法中选出最优方案，实现企业利润最大化。

针对问题一，我们建立了**序贯检验—假设检验相结合**的抽样方法。首先构建假设检验框架，设立两个对立的假设。然后根据题设进行错误概率设定。其次在每次抽样后根据当前样本中的合格品和不合格品数量计算对数似然比。通过逐次更新对数似然比，我们可以实时判断当前数据支持哪种假设，并决定是否继续抽样。之后计算出**对数似然比累计值**的阈值和停止准则，得到信度=95%时，**上限 $A=2.89$，下限 B= -2.25**。信度=90%时，**上限 A=2.20，下限 B= -2.20**。当对数似然比时≥A，拒收这批零配件。当对数似然比时≤B时，接受这批零配件。如果B ＜ 对数似然比 ＜ A，则继续检测。除此之外，我们利用 **z 检验**，计算出 Z 值。之后采用单侧检验来判断次品率与标称值的差异，得到在两个信度下的**正态分布临界值 Z_α**，通过比较 Z 值和 Z_α 的大小关系即可做出判断。同时进行误差分析，在设定允许误差=2%下，得到当信度=90%时当若样本量抽取总数达到 **369** 即可做出判断；当在信度=95%时样本量抽取总数达到 **609** 即可作出判断。

针对问题二，首先采用**蒙特卡洛模拟方法**。通过大量模拟随机试验，利用**梅森旋转随机数生成器**生成对应的随机数来判断检测品是否为次品。其次估计次品率在每个阶段的变化，并计算其最终平均值。再将每种选择翻译成二进制的形式，构建出每种决策的二进制表达，利用程序快速计算出在不同情况下每种决策的利润。最终得到 **10 种**不同决策方法在 **6 种**不同情况下的利润值，从而得到最优策略。

针对问题三，由于涉及到更复杂的多阶段生产过程，因此选择**遗传算法**作为求解工具。首先随机生成一个种群，用二进制变量定义决策。之后建立**适应度函数**，通过适应度函数评估每个个体后，使用**轮盘赌选择法**进行选择。随后先后进行**交叉操作**和**变异操作**，交叉操作选择使用**单点交叉**的方法。最后制定终止条件后，输出适应度值最高的个体，即企业利润最大的决策组合。

针对问题四，首先依据题设中的标称度利用梅森旋转随机数生成器随机生成大量零件数据，对每组零件进行**序贯检验逐次抽样**，得到新的次品率，输出次品率估计值。之后延续二三问的思路，分别用蒙特卡洛模拟方法和遗传算法计算出最优的检测与拆解方案以及利润最大的决策选择。

关键词：序贯检验　假设检验　蒙特卡洛模拟　遗传算法

一、 问题重述

某种电子产品是某企业的畅销产品，企业通过组装两种分别采购的零配件从而得到成品。企业对此产品有严格的检验标准：在装配的成品中，若其中一个零配件不达标，则成品一定不达标；若两个零配件均达标，装配出的成品也不一定达标。对于不达标成品，企业可以选择报废或对其拆解，拆解不会损坏零配件，但企业需要承担拆解费用。

了解了产品的检验要求，建立数学模型解答以下问题。

问题一：供应商承诺零配件 1 和零配件 2 的次品率不会超过 10%。企业计划采用抽样检测方法决定是否从此供应商购买这批零配件，检测费用由企业自行承担。为企业设计抽样检测方案以最大化减少检测次数从而降低成本。分别在下列两种情况下给出具体数据：

(1) 在 95%的信度下认定零配件次品率超过 10%，则拒收这批零配件；

(2) 在 90%的信度下认定零配件次品率不超过 10%，则接收这批零配件。

问题二：已知两种零配件和成品次品率，对企业生产过程的每个阶段进行决策：

(1) 是否对两种零配件进行检测，若某种零配件不进行检测，则此零配件将直接送入到装配环节；若检测出不合格的零配件则丢弃；

(2) 是否对装配完成的每一件成品进行检测，若不检测，装配后的成品直接进入到市场；否则只将检测合格的成品送入市场；

(3) 是否对检测出的不合格成品进行拆解，若不拆解，则将不合格成品直接丢弃；若进行拆解，则对拆解后的零配件重复步骤(1)和步骤(2)。

(4)，企业保证无条件对用户购入的不合格品予以调换，产生的调换损失（如物流成本、企业信誉等）由企业承担。对退回的不合格品重复步骤(3)。

将各项决策应用于表 1 中的实际情况，并提供决策的依据及相应的指标结果。

问题三：根据图 1 给出的 2 道工序、8 个零配件的情况，和表 2 给出的具体数值，基于已知零配件、半成品和成品的次品率，重复问题 2。给出具体的决策方案，以及决策的依据及相应指标。

问题四：假设问题 2 和问题 3 中零配件、半成品和成品的次品率均由抽样检测方法得来，重新解决问题 2 以及问题 3。

二、 问题分析

2.1 问题一的分析

针对问题一，我们首先想到利用**固定样本量**的**假设检验方法**。但我们发现假设检验方法具有一定局限性。其样本量是固定的，不能根据检测结果动态调整，可能导致样本量过大，从而使企业承担过多检测费用。

于是我们在此基础上进行优化，利用**序贯检验方法（SPRT）**，通过动态计算对数似然比来决定是否停止抽样检测。首先确定两个假设：原假设 H_0 和备择假设 H_1。之后在二项分布下推导出**对数似然公式（LLR）**。再根据题目要求，分别根据不同的

信度，得到显著性水平α和二类错误概率 1−β，依此计算出停止检验的两个**边界值**。即可在抽样过程中，对每个检测样本计算并累计对数似然比。若累计的对数似然比达到或超过上边界A，即拒绝原假设；如果累计的对数似然比达到或低于下边界B，即接受原假设。

利用序贯检验方法可以最大化减少检测次数，提高了抽样检测的上限。而假设检验方法也保证了若多次抽样后序贯检验的对数似然比仍位于上下边界之间，继续抽样得到的样本不会浪费，在检验次数接近企业期望上限时利用已有样本量得出结论，为企业最大化减低成本。

2.2 问题二的分析

第二问生产过程中各步骤的次品率变化复杂，难以通过解析方法直接计算，因此我们采用了**蒙特卡洛模拟方法**。通过模拟多次随机试验，利用**梅森旋转随机数生成器**生成对应的随机数来判断检测品是否为次品。估计次品率在每个阶段的变化，并计算其最终平均值。再将每种选择翻译成**二进制**的形式，构建出每种不同决策的二进制表达，利用程序快速计算出在不同情况下每种决策的利润，直观地比对利润大小，从而迅速定位最优决策。该方法能有效处理随机性强、过程复杂的情况，帮助我们快速得出合理的次品率估计结果，进而帮助选择决策选择。

2.3 问题三的分析

第三问涉及到更复杂的多阶段生产过程，随着零配件、半成品、成品数量的增加，决策空间的规模呈指数级增长。传统的优化方法难以有效求解如此大规模的组合优化问题。因此，选择**遗传算法**作为求解工具。

首先我们随机生成一个种群，随机生成多个初始解，用二进制变量定义决策。之后建立**适应度函数**，用于评估每个决策组合的表现。此过程需要我们推导出总成本的计算公式，最终求出各种决策的总利润。通过适应度函数评估每个个体后，使用**轮盘赌选择法**进行选择。轮盘赌选择根据个体的适应度值决定其被选中的概率，适应度值越高的个体被选中的概率越大。然后来到遗传算法的核心步骤，先后进行**交叉操作**和**变异操作**，交叉操作我们选择使用**单点交叉**的方法。遗传算法会不断重复选择、交叉和变异操作，因此我们制定了终止条件：当连续若干代的最优个体适应度值不再显著变化时，认为算法已经收敛，终止运行。最终输出的是适应度值最高的个体，即企业利润最大的决策组合。

2.4 问题四的分析

此问要求零配件、半成品和成品的次品率数据均需要通过抽样检测方法得来。

延续第一问的思路：依据题设中的标称度利用**梅森旋转随机数生成器**随机生成大量零件数据，对每组零件进行**序贯检验**逐次抽样，可得到新的次品率，最终输出次品率估计值。

对于第二问的重新求解，延续第二问的思路：利用**蒙特卡洛方法**生成大量随机数，对不同决策方案进行模拟，最终找出最优的检测与拆解方案，最小化总成本。

对于第三问的重新求解，延用遗传算法：依次完成初始化种群、建立适应性函数、轮盘赌选择、交叉、变异和适应度评估，输出应度值最高的个体，得到企业利润最大的决策组合。

三、 模型假设

1. 假设问题一中两种信度下第二类错误 β 均为 0.10。
2. 假设问题二中不合格的成品若选择拆解，则只拆解一遍。
3. 假设问题二中对不合格的成品进行拆解，前提两种零配件均经过检测且合格。
4. 假设问题三中每个成品或半成品最多只拆解一次。
5. 假设问题三中拆解后的产物不再进行检测。

四、 符号申明

本文涉及到的符号如下表所示。

表 1：本文主要涉及的符号说明

符号	符号含义及说明
α	显著性水平
β	第二类错误概率
Λ	对数似然比
p	样本次品率
A	序贯检测停止抽样的上限
B	序贯检测停止抽样的下限
Z	衡量样本统计量与总体参数之间标准化差异的统计量
Z_α	单侧检验下正态分布的临界值
c	成本
M	生产的成品总数
T	蒙特卡洛模拟的次数

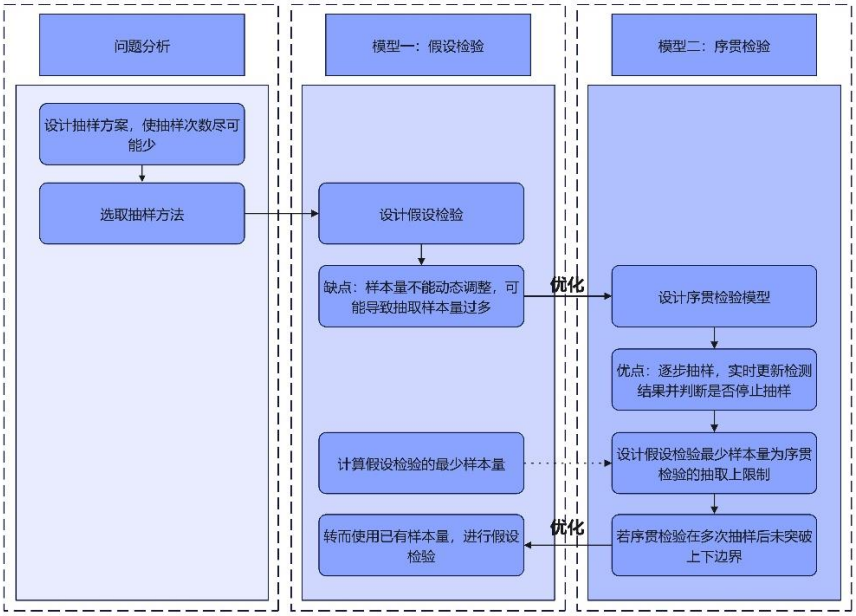
五、 模型的建立与求解

5.1 问题一的求解

5.1.1 模型的简要介绍

本模型采用了**序贯检验和假设检验相结合的方法**，首先利用序贯抽样检测方法，逐步抽样，实时更新检测结果并判断是否需要继续抽样，从而避免不必要的大规模检测，最大化减少检测次数。序贯抽样方法在检测效率和样本量控制上具有显著优势。为了避免序贯抽样无法及时得出结论的情况，模型还结合了传统的假设检验方法。在序贯抽样多次抽样后未能突破停止界限时，利用已有的样本量进行显著性检验，保证检测的效率和结果的准确性。

序贯检验—假设检验流程图



5.1.2 序贯检验模型的建立

1、首先构建假设检验框架，设立两个对立的假设，标称值由题目给定为 10%：

原假设H_0：次品率 p≤10%，即零配件的次品率低于标称值。

备择假设H_1：次品率＞10%，即零配件的次品率超过标称值。[2]

2、进行错误概率设定

第一种情形：信度 95%对应显著性水平 α=0.05。

第二种情形：信度 90%对应第二类错误概率 β=0.10。

3、计算样本的对数似然比

序贯检验的核心是计算每个样本的对数似然比，根据合格品或次品的检测结果累计计算，计算公式如下

$$\Lambda = \log\left(\frac{P(\text{数据} \mid H_1)}{P(\text{数据} \mid H_0)}\right) \tag{1}$$

其中：

$P(\text{数据} \mid H_1)$是在假设次品率p_1（备择假设）时，样本数据的概率。

$P(\text{数据} \mid H_0)$是在假设次品率为p_0（原假设）时，样本数据的概率。

通过逐次更新对数似然比，我们可以实时判断当前数据支持哪种假设，并决定是否继续抽样。

在每次抽样后，我们根据当前样本中的合格品和不合格品数量计算对数似然比，计算公式如下：

$$\mathrm{LLR}=\sum_{i=1}^{n}\left[x_i\log\left(\frac{p_1}{p_0}\right)+(1-x_i)\log\left(\frac{1-p_1}{1-p_0}\right)\right] \tag{2}$$

其中：

$x_i=1$表示检测到次品，$x_i=0$表示检测到合格品。

p_0 是原假设下的次品率，p_1 是备择假设下的次品率。

对每一个检测样本：

如果检测到次品（$x_i=1$），对数似然比的增量为：

$$\log\left(\frac{p_1}{p_0}\right) \tag{3}$$

如果检测到合格品（$x_i=0$），对数似然比的增量为：

$$\log\left(\frac{1-p_1}{1-p_0}\right) \tag{4}$$

4、计算阈值和停止准则

为了设定停止抽样条件，需要计算两个重要参数：**上限**和**下限**，分别用于判断何时停止检验，拒绝原假设 H_0 或接受备择假设 H_1。

上限A：当对数似然比时$\Lambda\geq A$，拒绝H_0，认为次品率超过标称值。

下限B：当对数似然比时$\Lambda\leq B$，接受H_0，认为次品率不超过标称值。

上限A的计算公式为：

$$A=\log\left(\frac{1-\beta}{\alpha}\right) \tag{5}$$

下限B的计算公式为：

$$B=\log\left(\frac{\beta}{1-\alpha}\right) \tag{6}$$

其中：

α 是显著性水平，表示第一类错误的概率，即错误地拒绝原假设H_0的概率。

$$\alpha=P(\text{拒绝 } H_0 \mid H_0 \text{ 为真})$$

对于情形 1，α=5%。对于情形 2，α=10%。

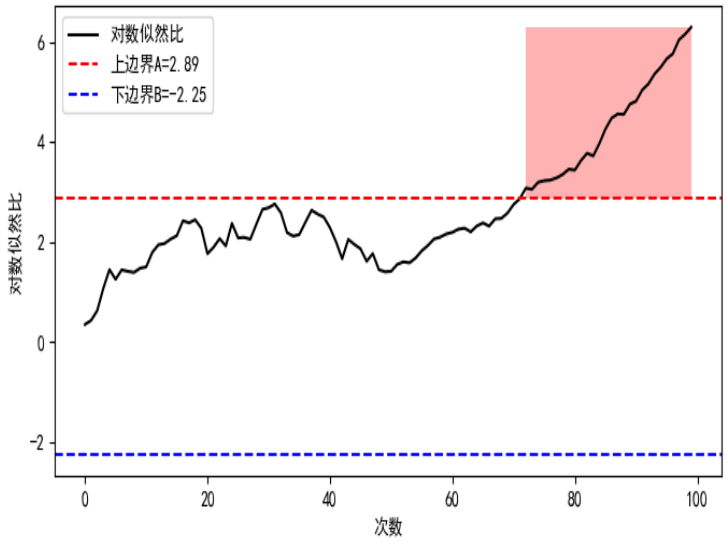
β 是第二类错误的概率，即错误地接受原假设H_0的概率。

$$\beta=P(\text{接受 } H_0 \mid H_1 \text{ 为真})$$

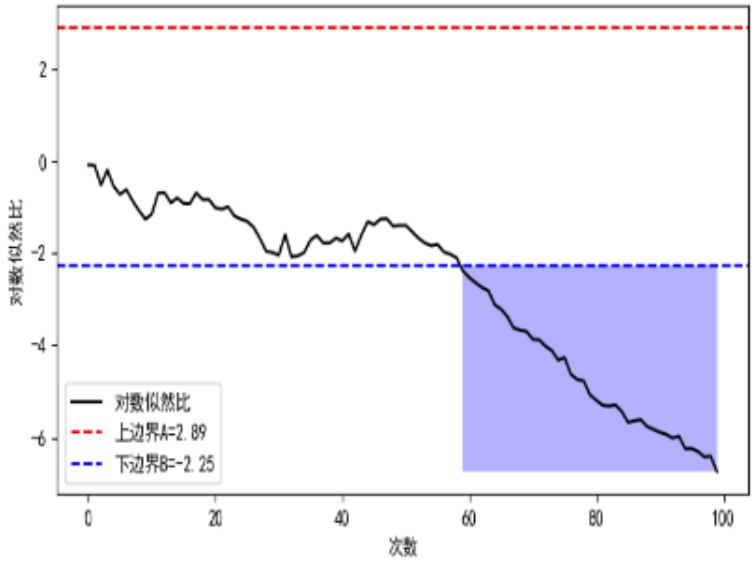
在两种情形中 β 均设为 10%。

使用对数似然比逐步更新抽样结果，当对数似然比超过上限或者下限，则停止抽样，根据结果可以判断对这批零配件接受或者拒收，作出决策。

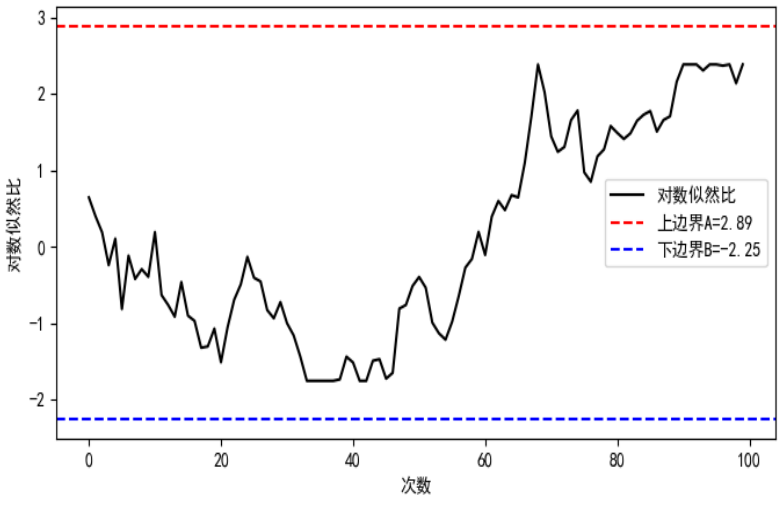
- *如果对数似然比时$\Lambda \geq A$，拒收这批零配件。*



- *如果对数似然比时$\Lambda \leq B$，接受这批零配件。*



- *如果$B <$ 对数似然比$< A$，继续检测。*



5、结合假设检验

如果序贯检验在多次抽样后未突破上下边界，转而使用已有样本量进行假设检验。此时，可以基于当前样本的次品率 p 和总样本量 n 进行 z 检验，判断次品率是否显著高于或低于标称值，以确保能够利用现有的样本量作出合理的决策。还可利用假设检验计算出需要的样本量。

5.1.3 假设检验模型的建立

在抽样检测中，z 检验是一种用于判断样本数据是否与假设的总体参数有显著差异的统计方法。z 检验通常用于样本量较大且数据接近正态分布的场合。当序贯检验多次抽样后未能突破上下边界时，我们将转而使用 **z 检验**对已有的样本进行显著性检验，以确保能够利用现有的样本量作出合理的决策。[1]

1、假设设定

Z 检验模型的假设设定同序贯检验，假设如下：

原假设H_0：次品率 p≤10%，即零配件的次品率低于标称值。

备择假设H_1：次品率＞10%，即零配件的次品率超过标称值。

2、检验方法

在本问题中，我们采用单侧检验来判断次品率与标称值的差异

3、计算 Z 值

$$Z = \frac{\hat{p} - p_0}{\sqrt{\frac{p_0(1-p_0)}{n}}} \tag{7}$$

其中样本次品率为：

$$\hat{p} = \frac{x}{n} \tag{8}$$

x为样本中次品的数量，n为样本量，p_0是标称次品率=10%

分母是标准误差，反映了样本次品率与总体次品率之间的标准化差异。

4、判断标准：

在单侧检验中，我们只关心次品率是否**高于**标称值。因此，我们只需考虑单侧的显著性水平 α。

对于单侧检验，正态分布的临界值Z_α表示在给定的显著性水平 α 下，拒绝原假设的标准化差异的界限。

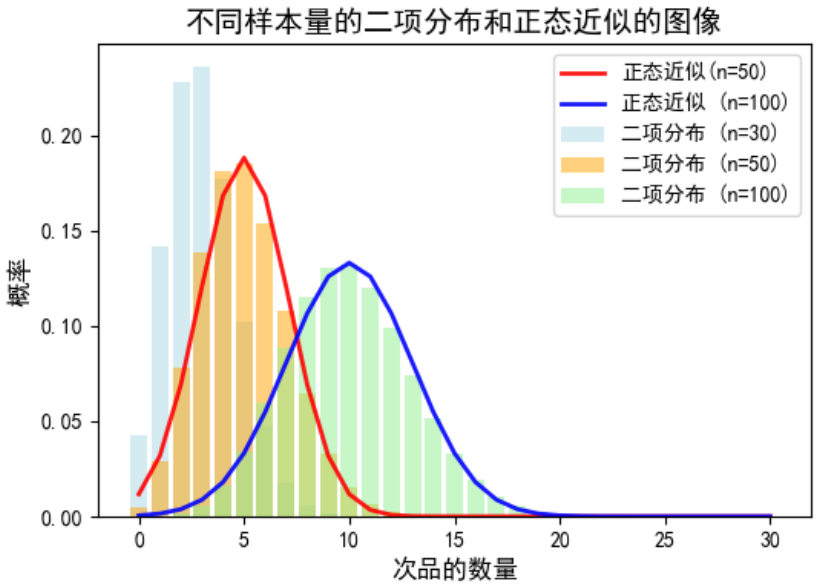
$$Z_\alpha = \Phi^{-1}(1-\alpha) \tag{9}$$

其中：

Φ^{-1}是标准正态分布的反函数。

- 如果$Z > Z_\alpha$，我们有足证据拒绝原假设H_0，认为次品率显著高于标称值；
- 如果$Z \leq Z_\alpha$，则没有足够证据拒绝原假设H_0，认为次品率未显著高于标称值。

在样本量足够大的时候，二项分布的图像与正态分布无限接近。由图像可知，当 n≥50 时，二项分布已经十分接近于正态分布。因此本题可利用 z 检验。



5、误差限制

此模型允许企业根据实际情况设置误差限制 E，为了控制样本次品率回答总体次品率之间的误差不超过设定的误差限 E，可以得出误差 E 与抽样总计样本量的关系。即可在给定的置信水平下，通过设置误差 E，求得所需抽样的样本量 n。

不等式如下：

$$Z_\alpha \cdot \sqrt{\frac{p(1-p)}{n}} \leq E \tag{10}$$

对不等式进行代数变化，即可推导出样本量公式：

$$n = \frac{Z_\alpha^2 \cdot p(1-p)}{E^2} \tag{11}$$

假设允许误差 E=2%

- 对于 95%的置信区间（单侧检验，$Z_\alpha = 1.645$）

$$n = \frac{2.706 \cdot 0.1 \cdot 0.9}{0.0004} = \frac{0.24354}{0.0004} = 609$$

因此，当样本量抽取总数达到 609，即可作出判断。

- 对于 90%的置信区间（单侧检验，$Z_\alpha = 1.28$）

$$n = \frac{1.6384 \cdot 0.1 \cdot 0.9}{0.0004} = \frac{0.147456}{0.0004} = 369$$

因此，当样本量抽取总数达到 369，即可作出判断。

5.2 问题二的求解

5.2.1 模型的简要介绍

根据第二问的题设，一共会衍生出 2^4 种决策。但我们实际操作中只考虑 8 种决策，原因是我们认为只有在两种零配件都经过检测且检测合格的情况下，对不合格成品进行拆解才能节约成本。故在实际选择中，我们只考虑零配件一、二均经检测合格才考虑对不合格成品进行拆解，即 10 种决策。我们基于给定的次品率、检测成本、装配成本、拆解成本等因素，利用蒙特卡洛方法过随机数模拟来帮助决策。对不同决策方案进行模拟，从而找出最优的检测与拆解方案，以最小化总成本。[3]

5.2.2 模型假设

基于本问，为了简化问题并合理建立模型，提出以下假设：

1. 结合生产实际，对零配件多次反复拆解会造成零件损耗，影响成品质量。所以我们假设检测不合格的成品若选择拆解，则只拆解一遍。
2. 对不合格的成品进行拆解，前提两种零配件均经过检测且检测合格。此举意在减少拆解成本，同时有效简化模型，更利于选择出最优决策。

5.2.3 蒙特卡洛方法的符号说明

表 2：蒙特卡洛方法涉及的符号说明

符号	符号含义及说明
M	生产的成品总数
p_1, p_2	分别表示零配件 1 和零配件 2 的次品率
p_f	成品的次品率
wa, wb, wc	分别表示零配件 1、零配件 2 以及装配成品的成本
ca, cb, cc	分别表示零配件 1、零配件 2 以及成品的检测成本
r	成品的市场售价
c_d	成品拆解的成本
c_r	不合格成品的调换损失
d[i]	表示企业决策
T	模拟的次数

其中：

d[i]为决策变量，包含是否检测零配件 1、零配件 2、成品，以及是否对不合格成品进行拆解和退换，分别用 $d[i] \in \{0,1\}$表示：

d[1]：是否检测零配件 1

d[2]：是否检测零配件 2

d[3]：是否检测成品

d[4]：是否拆解不合格成品

5. 2. 4 蒙特卡洛方法的模型建立

step1：

进行随机样本生成，对于每一轮模拟，利用随机数生成零配件 1、零配件 2 以及成品的质量：

X_1, X_2是 0-1 随机变量，表示零配件 1 和零配件 2 是否合格，装配后的成品质量Y同样为 0-1 随机变量

对于每一轮模拟，根据零配件 1、零配件 2 以及成品的次品率，我们利用**梅森旋转随机数生成器**生成对应的随机数来判断是否为次品。

设 rnd 为通过随机数生成器生成的 0 到 100 之间的整数（用于模拟百分比概率），

具体生成方式如下：

```
int rnd = mt() % 100 + 1;
```

其中，mt()生成的是一个非常大的整数，经过取模操作后，其范围被缩减为 0 到 99，再通过加 1 操作，最终rnd的取值范围为 1 到 100 的整数，用于模拟 0%到 100%之间的次品率。

对于M件成品，每件成品的零配件 1、零配件 2 和成品的质量状态通过以下规则确定：

- 零配件 1 的状态：

$$若\ rnd \le p_1 \times 100，则X_1 = 0(次品)$$

$$若\ rnd > p_1 \times 100，则X_1 = 1(合格品)$$

- 零配件 2 的状态：

$$若\ rnd \le p_2 \times 100，则X_2 = 0(次品)$$

$$若\ rnd > p_2 \times 100，则X_2 = 1(合格品)$$

- 成品的状态：

$$若\ rnd \le p_f \times 100，则Y = 0(次品)$$

$$若\ rnd > p_f \times 100，则Y = 1(合格品)$$

step2:

1. **零配件检测**

根据 $d[1],d[2]$ 判断是否对零配件 1 和零配件 2 进行检测。

如果 $d[1]=1$，

检测零配件 1，若为次品，则直接丢弃，计算检测成本 ca

如果 $d[2]$=1，检测零配件 2，若为次品，则直接丢弃，计算检测成本 cb

如果不检测，则直接将零配件送入装配环节。

2. 成品装配

如果零配件 1 和零配件 2 合格，继续装配成品，装配成本为 wc

3. 成品检测

根据 $d[3]$ 判断是否对成品进行检测。

若 $d[3]$=1，检测成品。
若检测结果为次品，则根据 $d[4]$判断是否对次品进行拆解。

若 $d[3]$=0，则成品直接进入市场。

4. 不合格成品的拆解与处理

如果成品检测为次品，根据 $d[4]$决策：

若 $d[4]$=1，对次品进行拆解，拆解成本为 cd，并将拆解出的零配件送回零配件检测流程重新使用。

若 $d[4]$=0，则次品丢弃或根据 判断是否进行退换。若不退换，产生调换损失 cr。

step3:

每一轮模拟后，计算当前轮次的总成本与收益对于每一次生产模拟，
成本包括：

- 零配件检测成本：

$$C_{\text{零配件}} = d[1] \cdot M \cdot ca + d[2] \cdot M \cdot cb \tag{12}$$

其中 d[1] 和 d[2] 表示是否对零配件进行检测。

- 成品检测成本：

$$C_{\text{成品}} = d[3] \cdot M \cdot cc \tag{13}$$

若 $d[3]$=1，表示检测成品，检测成本为$M \cdot cc$。

- 成品装配成本：

$$C_{\text{装配}} = M \cdot wc \tag{14}$$

所有成品的装配成本为 wc。

- 拆解费用：

$$C_{\text{拆解}} = d[4] \cdot M \cdot c_d \tag{15}$$

- 调换损失：

$$C_{\text{调换}} = (1 - Y) \cdot M \cdot c_r \tag{16}$$

- 总成本：

$$C_{\text{总}} = C_{\text{零配件}} + C_{\text{成品}} + C_{\text{拆解}} + C_{\text{调换}} \tag{17}$$

- 每轮的净利润为：

$$\begin{aligned} S &= r \cdot M \\ \text{利润} &= S - C_{\text{总}} \end{aligned} \tag{18}$$

step4:

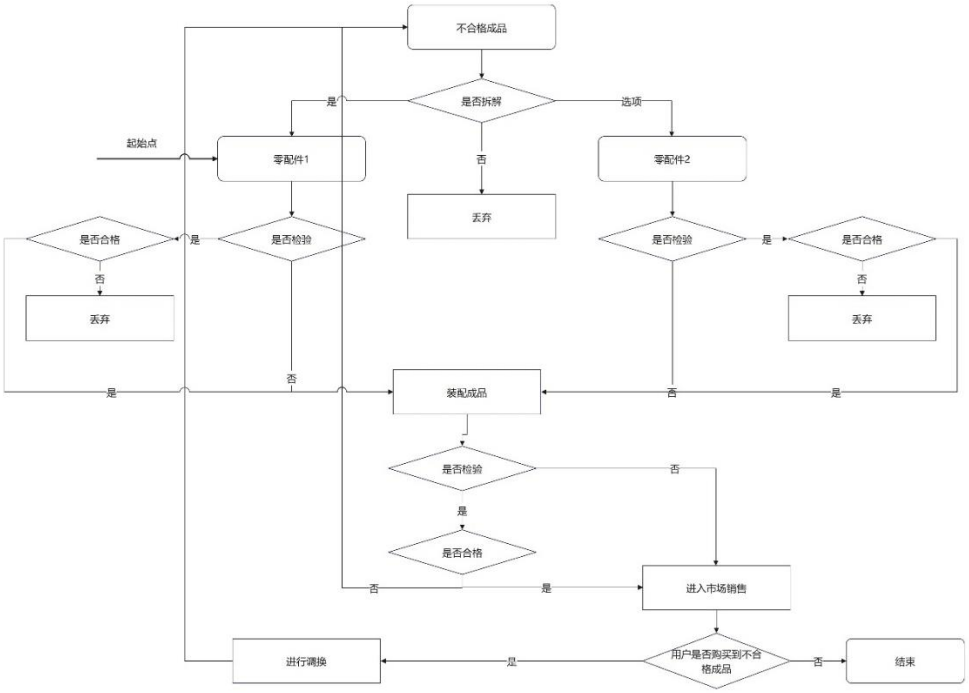
进行多轮模拟与结果统计，重复上述步骤 T次，每次记录利润。

在完成所有模拟后，计算出所有轮次的平均利润：

$$\bar{\text{利润}} = \frac{1}{T}\sum_{i=1}^{T} \text{利润}_i \tag{19}$$

通过计算不同决策方案下的平均利润，作为评估指标。最终通过统计大量模拟结果，我们可以找到最优的检测和拆解决策方案。

完整的决策图如下：



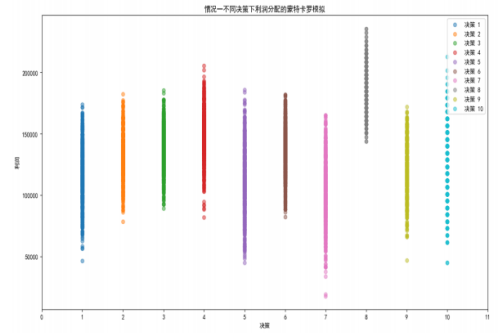
5.2.5 模型的求解及决策展示

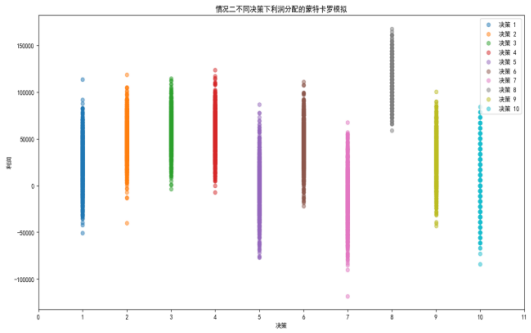
使用 c++进行运算，计算总计 10 种决策下，在企业生产遇到的 6 种不同情况下的利润，直观地比对不同情况下的最优决策。

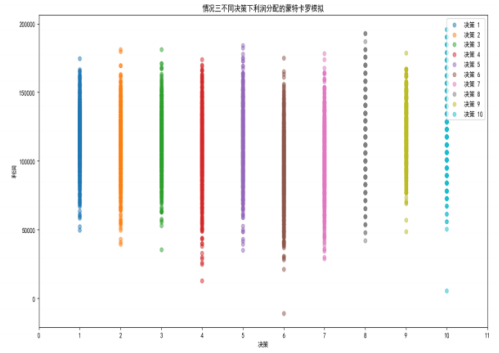
分别六种情况下的最优决策

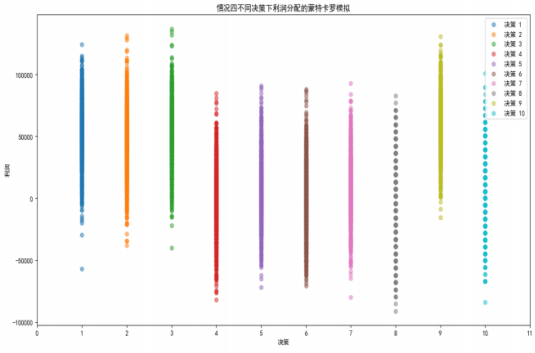
	零件一检测	零件二检测	成品检测	拆解	利润（元）
情况一	0	0	0	0	189866
情况二	1	1	0	1	128962
情况三	1	1	1	1	163155
情况四	1	1	1	1	123584
情况五	0	0	0	0	148330
情况六	0	0	0	0	226078

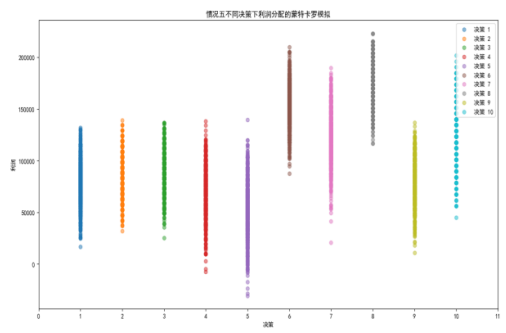
下图为蒙特卡洛假设的六种情况的可视化

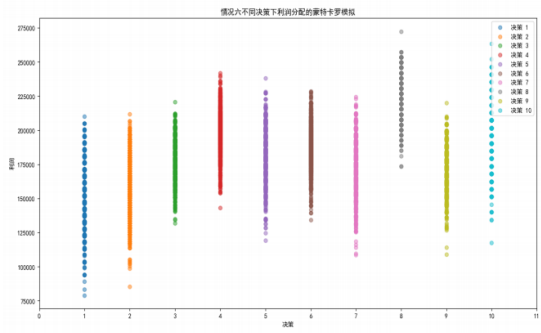












下面依次展示六种情况的利润比较：

表 3：企业生产中的第一种情况

零件一检测	零件二检测	成品检测	拆解	利润（元）
1	1	1	1	163155
1	1	1	0	129402
1	1	0	1	182455
1	1	0	0	146842
1	0	0	0	150748
1	0	1	0	112062
0	1	0	0	139319
0	1	1	0	102733
0	0	1	0	123591
0	0	0	0	189866

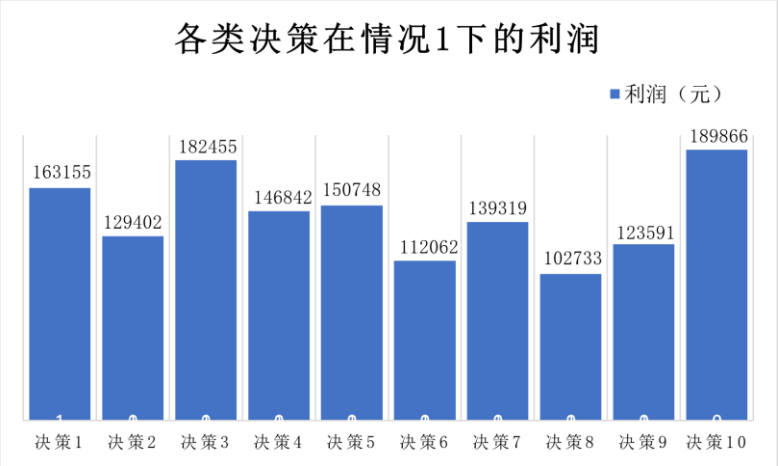


表 4：企业生产中的第二种情况

零件一检测	零件二检测	成品检测	拆解	利润（元）
1	1	1	1	91722
1	1	1	0	38924
1	1	0	1	128962
1	1	0	0	70204
1	0	0	0	60014
1	0	1	0	-5559
0	1	0	0	48414
0	1	1	0	-12702
0	0	1	0	222
0	0	0	0	118772

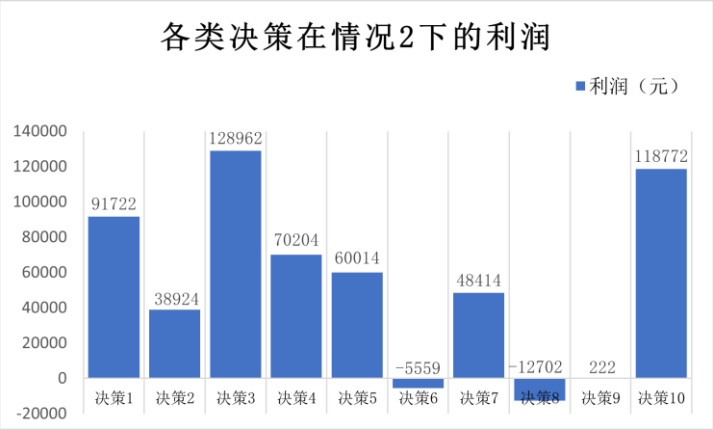


表 5：企业生产中的第三种情况

零件一检测	零件二检测	成品检测	拆解	利润（元）
1	1	1	1	163155
1	1	1	0	129402
1	1	0	1	159214
1	1	0	0	125914
1	0	0	0	110380
1	0	1	0	112062
0	1	0	0	99407
0	1	1	0	102733
0	0	1	0	123591
0	0	0	0	126242



表 6：企业生产中的第四种情况

零件一检测	零件二检测	成品检测	拆解	利润（元）
1	1	1	1	123584
1	1	1	0	70488
1	1	0	1	119860
1	1	0	0	67360
1	0	0	0	4441

1	0	1	0	7292
0	1	0	0	7298
0	1	1	0	10076
0	0	1	0	4964
0	0	0	0	4964

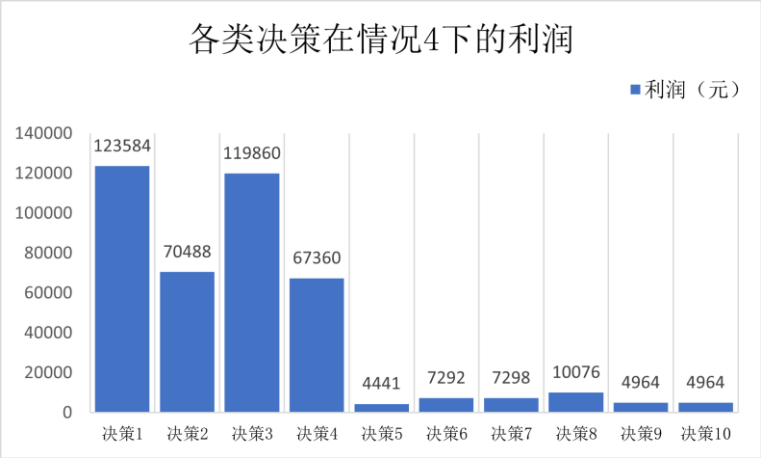


表 7：企业生产中的第五种情况

零件一检测	零件二检测	成品检测	拆解	利润（元）
1	1	1	1	76967
1	1	1	0	45554
1	1	0	1	86383
1	1	0	0	54211
1	0	0	0	36484
1	0	1	0	6532
0	1	0	0	112973
0	1	1	0	84606
0	0	1	0	79030
0	0	0	0	148330

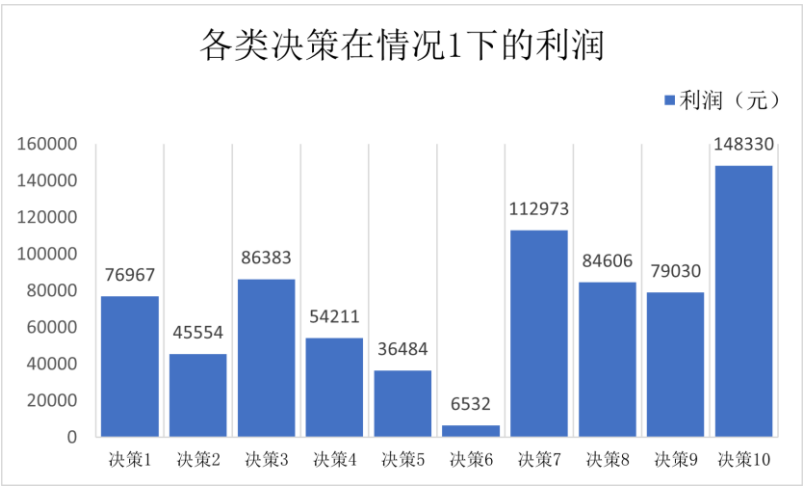
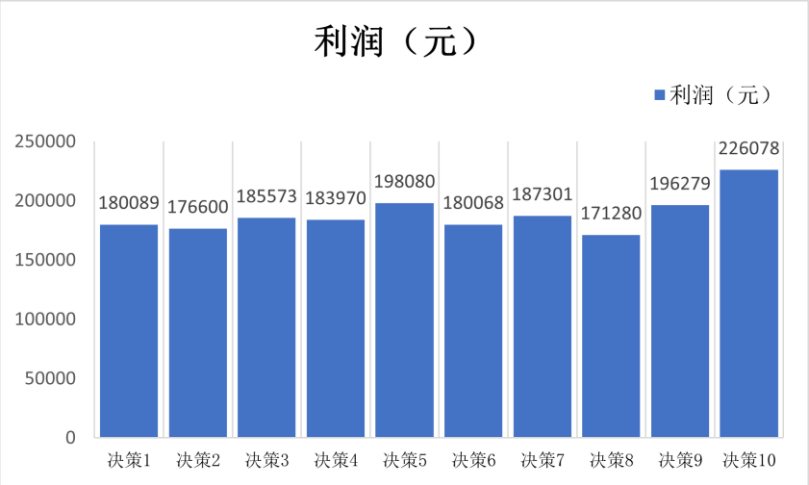


表 8：企业生产中的第六种情况

零件一检测	零件二检测	成品检测	拆解	利润（元）
1	1	1	1	180089
1	1	1	0	176600
1	1	0	1	185573
1	1	0	0	183970
1	0	0	0	198080
1	0	1	0	180068
0	1	0	0	187301
0	1	1	0	171280
0	0	1	0	196279
0	0	0	0	226078



5. 3 问题三的求解

5. 3. 1 问题背景：

第三问涉及到更复杂的多阶段生产过程，随着零配件、半成品、成品数量的增加，决策空间的规模呈指数级增长。传统的优化方法难以有效求解如此大规模的组合优化问题。因此，选择**遗传算法**作为求解工具，能够利用其强大的全局搜索能力在高维复杂空间中找到近似最优解。

5. 3. 2 模型假设

基于本问，为了简化问题并合理建立模型，提出以下假设：

1. 每个成品或半成品最多只拆解一次。

   原因如下：

   出于实际考虑，反复拆解会造成零配件损耗。多次拆解也会衍生出无限轮的决策和选择，使得模型臃肿庞杂，无法找到近似最优解。

2. 拆解后的产物不再进行检测。

原因如下：

若组成产物的零配件已经检测过，则显然组成成品的零配件均检测合格，于是没有再次检测零配件的必要。

为贯彻简化模型、帮助企业节约决策时间和成本的目的，若第一轮生产中组成产物的零配件没有经过检验，那第二轮生产再进行检验只会增加检验成本且效果显著差于在上一轮检验，不再具备检测的必要性。

5.3.3 遗传算法的符号说明

表 9：遗传算法涉及的符号说明

符号	符号含义及说明
p_i	第 i 个零配件、半成品或成品的次品率
$c_{\text{inspect},i}$	检测第 i 个零配件、半成品或成品的成本
$c_{\text{assembly},i}$	装配第 i 个零配件、半成品或成品的成本
$c_{\text{disassembly},i}$	拆解不合格半成品或成品的成本
c_{return}	不合格成品被退换时的损失
p_{final}	成品的市场售价
n	生产的成品或零件数量

其中：

x_i为决策变量，是否对第 i 个零配件、半成品或成品进行检测，$x_i = 1$ 表示进行检测，$x_i = 0$ 表示不检测。$x_i \in \{0,1\}$

di为决策变量，是否对不合格的第 i 个半成品或成品进行拆解，$di = 1$ 表示进行拆解，$di = 0$ 表示不拆解。$d[i] \in \{0,1\}$

5.3.4 遗传算法的模型建立

总成本由检测、装配、拆解和退换成本组成

零配件阶段成本：

在零配件阶段，企业决定是否对每个零配件进行检测。检测减少零配件的次品率，但会产生检测成本。零配件的总成本为：

$$C_{\text{零配件}} = \sum_{i=1}^{8} \left(x_i \cdot c_{\text{inspect},i} + c_{\text{assembly},i} \right) \cdot n \tag{20}$$

其中，$x_i = 1$ 表示对零配件 i 进行检测。

半成品阶段成本：

半成品由三个零配件装配而成，半成品的次品率是组成零配件次品率的乘积。半成品的成本包括检测成本和装配成本：

$$C_{\text{半成品}} = \sum_{i=9}^{11} \left(x_i \cdot c_{\text{inspect},i} + c_{\text{assembly},i}\right) \cdot n \tag{21}$$

成品阶段成本：

成品由半成品组装而成，成品的次品率由半成品次品率决定。成品的总成本为：

$$C_{\text{成品}} = (x_{12} \cdot c_{\text{inspect},12} + c_{\text{assembly},12}) \cdot n \tag{22}$$

拆解后的处理：

拆解后的产物直接进入下一步装配或销售环节，不再进行检测。拆解成本计算为：

$$C_{\text{拆解}} = \sum_{i=13}^{15} d_i \cdot c_{\text{disassembly,semi},i} + d_{16} \cdot c_{\text{disassembly,final}} \tag{23}$$

其中，di =1 表示拆解不合格的半成品或成品，拆解后不再进行检测。

退换成本：

不合格成品进入市场会产生退换损失，退换损失由次品率和市场售价决定。退换成本为：

$$C_{\text{退换}} = p_{12} \cdot c_{\text{return}} \cdot n \tag{24}$$

总成本：

将以上各部分相加得到：

$$\begin{aligned} C_{\text{总}} &= C_{\text{零配件}} + C_{\text{半成品}} + C_{\text{成品}} + C_{\text{拆解}} + C_{\text{退换}} \\ &= \sum_{i=1}^{8} \left(x_i \cdot c_{\text{insyect},i} + c_{\text{sasemulby},i}\right) \\ &+ \sum_{i=0}^{11} \left(x_i \cdot c_{\text{inspec},i} + c_{\text{assemlby},i}\right) \cdot n \\ &+ \left(x_{12} \cdot c_{\text{inspect},i2} + c_{\text{assemlby},i2}\right) \cdot n + \sum_{i=13}^{15} d_i \cdot c_{\text{disasembly,wem,i}i} + d_{16} \\ &\cdot c_{\text{disasemby,final}} + p_{12} \cdot c_{\text{return}} \cdot n \end{aligned} \tag{25}$$

总利润：

$$S = p_{\text{final}} \cdot n$$
$$\text{利润} = S - (C_{\text{零配件}} + C_{\text{半成品}} + C_{\text{成品}} + C_{\text{拆解}} + C_{\text{退换}}) \tag{26}$$

5.3.5 遗传算法模型的求解

遗传算法的核心操作包括初始化种群、选择、交叉、变异和适应度评估。[4]具体步骤为：

step1：初始化种群

首先，随机生成一个种群，其中每个个体代表一个具体的决策方案。每个个体由一组二进制编码组成，包括检测和拆解的决策变量：

前 12 位：对应检测决策 $x1$， $x2$,…, $x12$ ，其中 $xi \in \{0,1\}$，表示是否对零配件、半成品或成品进行检测。

后 4 位：对应拆解决策 $d13$, $d14$, $d15$, $d16$ ，其中 $di \in \{0,1\}$，表示是否对半成品或成品进行拆解。

种群大小为 P ，随机生成 10000 个个体，每个个体的初始状态为随机生成的二进制串。

$$\mathbf{G}_0 = \{\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_P\}, \mathbf{x}_i \in \{0,1\}^{16}$$

其中， xi 是第 i 个个体的决策变量组合，表示一个具体的检测和拆解方案。

step2：适应度函数评估

适应度函数 $f(x)$ 用来评估每个个体（即每个决策组合）的表现。我们的目标是最大化企业的总利润。因此，适应度函数即为利润 Profit(x) ，即总收入减去总成本。

总收入公式为：

$$S = p_{\text{final}} \cdot n \tag{27}$$

得到了总收入公式和总成本公式，即可计算总利润：

$$f(\mathbf{x}) = \text{Profit}(\mathbf{x}) = S - C_{\text{总}}(\mathbf{x}) \tag{28}$$

对于每个个体 xi ，计算其适应度值 $f(xi)$ ：

$$f(\mathbf{x}_i) = p_{\text{final}} \cdot n - C_{\text{总}}(\mathbf{x}_i) \tag{29}$$

step3：选择操作

通过适应度函数评估每个个体后，使用轮盘赌选择法进行选择。轮盘赌选择根据个体的适应度值决定其被选中的概率，适应度值越高的个体被选中的概率越大。

令种群中所有个体的适应度值之和为 $\sum_{i=1}^{P} f(\mathbf{x}_i)$，每个个体 xi 被选中的概率为：

$$P(\mathbf{x}_i) = \frac{f(\mathbf{x}_i)}{\sum_{i=1}^{P} f(\mathbf{x}_i)} \tag{30}$$

通过随机抽取的方式，选取 P 个个体组成下一代。

step4：交叉操作

选择后的个体通过交叉操作产生新个体。交叉操作模拟生物中的基因重组，使得新个体继承父代个体的部分特征。

单点交叉：在父代的二进制编码中随机选择一个交叉点 k ，并交换两个个体在该点后的基因。

假设父代个体 $\mathbf{x}_a = [x_{a1}, x_{a2}, \dots, x_{a16}]$ 和 $\mathbf{x}_b = [x_{b1}, x_{b2}, \dots, x_{b16}]$ ，随机选定交叉点 k ，生成两个新个体 x1 和 x2 ：

$$\mathbf{x}_1 = [x_{a1}, x_{a2}, \dots, x_{ak}, x_{b(k+1)}, \dots, x_{b16}]$$

$$\mathbf{x}_2 = [x_{b1}, x_{b2}, \dots, x_{bk}, x_{a(k+1)}, \dots, x_{a16}]$$

step5: 变异操作

变异操作模拟生物进化中的基因突变，以保持种群的多样性并避免陷入局部最优。变异概率为 Pmut。

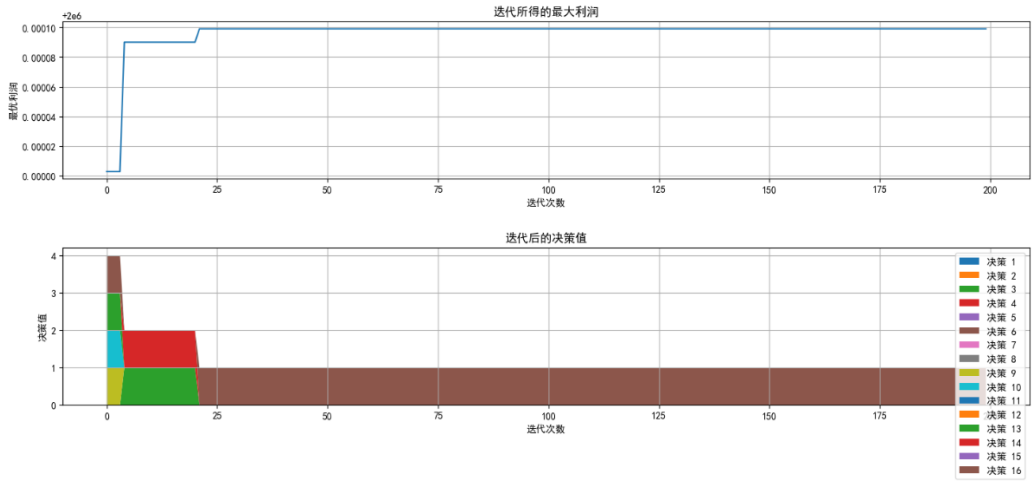
对于每个个体的每个位 xi ，以概率 Pmut 随机改变其值：

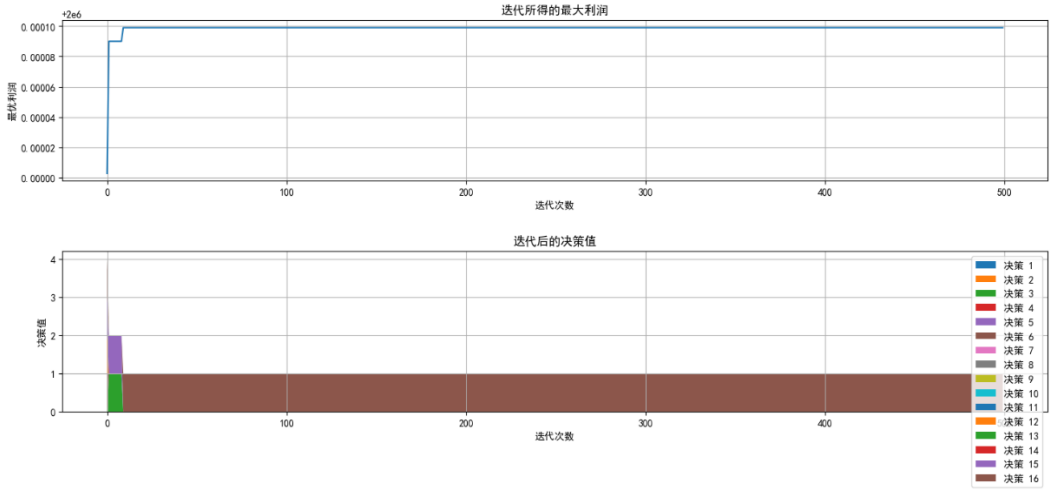
$$x_i = 1 - x_i(\text{如果发生变异})$$

通过变异，个体的某些决策可能会被随机改变，从而增加新的解的探索空间。

遗传算法不断重复选择、交叉和变异操作，直到当连续若干代的最优个体适应度值不再显著变化时，认为算法已经收敛，终止运行。

结果如下图所示：





由图可知决策 16 为最优决策

其决策为：

不检验零件一、不检验零件二、不检验成品、对不合格成品拆解

除此之外，我们尝试更改不同迭代次数，其结果由图 1 图 2 所示，最终结果相同。

5.4 问题四对问题二的重新求解

5.4.1 问题背景

此问要求零配件、半成品和成品的次品率数据需要通过抽样检测方法得来，根据我们第一问的解决方案，依据标称度生成 10000 组零件数据，对每组零件进行序贯检验逐次抽样，可得到 10000 个新的次品率，最终输出次品率估计值。再延续第二问的思路，利用蒙特卡洛方法生成大量随机数，对不同决策方案进行模拟，最终找出最优的检测与拆解方案，最小化总成本。

5.4.2 求解过程

step1：数据生成

表格中的给定的次品率是在生产过程中次品率的初始设定值，即为标称度 p。利用蒙特卡洛方法，通过标称度p 生成 10000 个零件数据。其中，标称度决定了零件数据的生成。在模拟过程中，所有零件数据的生成是基于这个标称度的，即假设零件的次品率等于标称度 p。

具体来说，每组**零件数据**遵循二项分布 Xi $\sim B(n,p)$，其中 p 决定了次品率。因此，标称度完全决定了生成的零件数据的统计性质。

step2：序贯检验模型

之后对每组零件数据 Xi 进行序贯检验，检验模型的核心为对数似然比的更新：

$$\Lambda = \log\left(\frac{L(p_1)}{L(p_0)}\right) = \log\left(\frac{p_1^{x_i}(1-p_1)^{n-x_i}}{p_0^{x_i}(1-p_0)^{n-x_i}}\right)$$

根据序贯检验的原理，设定上、下限分别为

$$A = \log\left(\frac{1-\beta}{\alpha}\right)$$

$$B = \log\left(\frac{\beta}{1-\alpha}\right)$$

通过逐步抽样和计算对数似然比，判断是否接受 $p0$ 或 $p1$，或者继续抽样。

每一组样本通过序贯检验得到一个新的次品率 pi，表示对原标称度 p 的估计。共计得到 10000 个新的次品率 pi。

其中序贯检验参数也由标称度设定。这是因为序贯检验中的两个核心参数 $p0$ 和 $p1$ 分别是检验过程中假设的“合格次品率”和“不合格次品率”。这两个参数是直接由标称度 p 设定的——即标称度确定了我们进行检验时假设的合格与不合格的标准。此外，检验的显著性水平 α和检验力 β 也保持固定不变。因此，整个序贯检验过程的设定是基于标称度进行的。

step3：误差处理和平均

每组次品率的误差较小，并且次品率在数轴上聚集在某一特定值附近，因此我们可以将这 10000 组新的次品率 pi
进行平均，得到该标称度对应的新的次品率 $p$new，即：

$$\hat{p}_{\text{new}} = \frac{1}{1000}\sum_{i=1}^{1000}\hat{p}_i \tag{31}$$

在这个模型中，标称度是影响零件数据生成和序贯检验过程的唯一变量，而其他参数（如 $p0$、$p1$、α、β）保持固定。因此，每个标称度 p 都会生成特定的零件数据，通过序贯检验得到唯一的修正次品率 $p$new。由于这个过程的确定性，标称度p和新次品率$p$new 之间是一一对应的。

虽然每次模拟中的次品率可能有轻微的波动和误差，但由于我们取了大量样本（10000 组零件数据），这些误差在序贯检验后趋于平均，误差较小且聚集在一个特定的数值附近。因此，最终的平均次品率 p^new 可以视为标称度p的稳定修正结果。

结果如下：

表 9：企业生产中的第一种情况

决策方案	零件一检测	零件二检测	成品检测	拆解	利润（元）
决策 1	1	1	1	1	167398
决策 2	1	1	1	0	136467
决策 3	1	1	0	1	184658
决策 4	1	1	0	0	152207
决策 5	1	0	0	0	158468
决策 6	1	0	1	0	123692
决策 7	0	1	0	0	147901
决策 8	0	1	1	0	113207
决策 9	0	0	1	0	134683
决策 10	0	0	0	0	196258

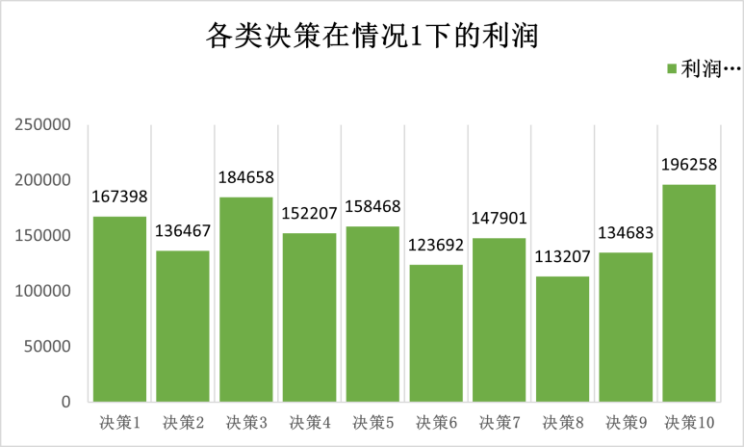


表 10：企业生产中的第二种情况

决策方案	零件一检测	零件二检测	成品检测	拆解	利润（元）
决策 1	1	1	1	1	95272
决策 2	1	1	1	0	43472
决策 3	1	1	0	1	130312
决策 4	1	1	0	0	73312
决策 5	1	0	0	0	62716
决策 6	1	0	1	0	-396
决策 7	0	1	0	0	50789
决策 8	0	1	1	0	-10877
决策 9	0	0	1	0	2700
决策 10	0	0	0	0	120200

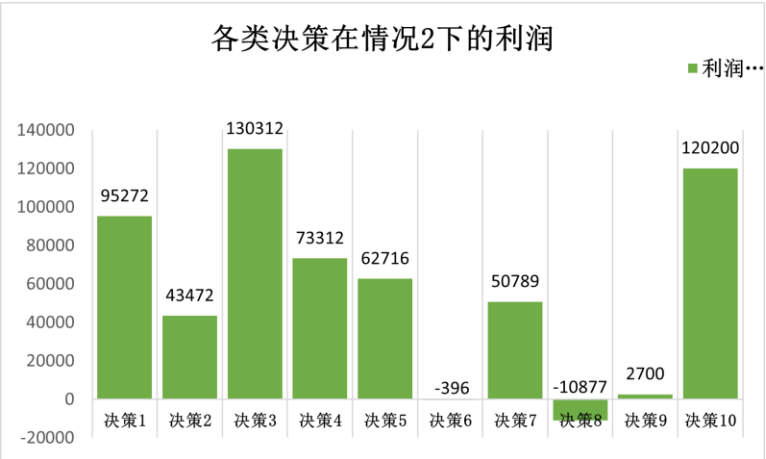


表 11：企业生产中的第三种情况

决策方案	零件一检测	零件二检测	成品检测	拆解	利润（元）
决策 1	1	1	1	1	167398
决策 2	1	1	1	0	136467
决策 3	1	1	0	1	163946
决策 4	1	1	0	0	133319
决策 5	1	0	0	0	122180
决策 6	1	0	1	0	123692
决策 7	0	1	0	0	110053
决策 8	0	1	1	0	113207
决策 9	0	0	1	0	134683
决策 10	0	0	0	0	137146



表 12：企业生产中的第四种情况

决策方案	零件一检测	零件二检测	成品检测	拆解	利润（元）
决策 1	1	1	1	1	127024
决策 2	1	1	1	0	74964
决策 3	1	1	0	1	123520
决策 4	1	1	0	0	71980
决策 5	1	0	0	0	9604
决策 6	1	0	1	0	12348
决策 7	0	1	0	0	9123
决策 8	0	1	1	0	11926
决策 9	0	0	1	0	7400
决策 10	0	0	0	0	7400

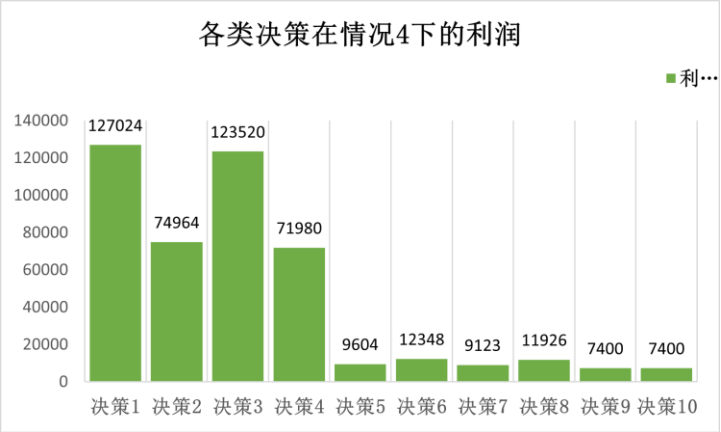


表 13：企业生产中的第五种情况

决策方案	零件一检测	零件二检测	成品检测	拆解	利润（元）
决策 1	1	1	1	1	81983
决策 2	1	1	1	0	53146
决策 3	1	1	0	1	90442
决策 4	1	1	0	0	60989
决策 5	1	0	0	0	44662
决策 6	1	0	1	0	16426
决策 7	0	1	0	0	119510
决策 8	0	1	1	0	92720
决策 9	0	0	1	0	87962
决策 10	0	0	0	0	154182

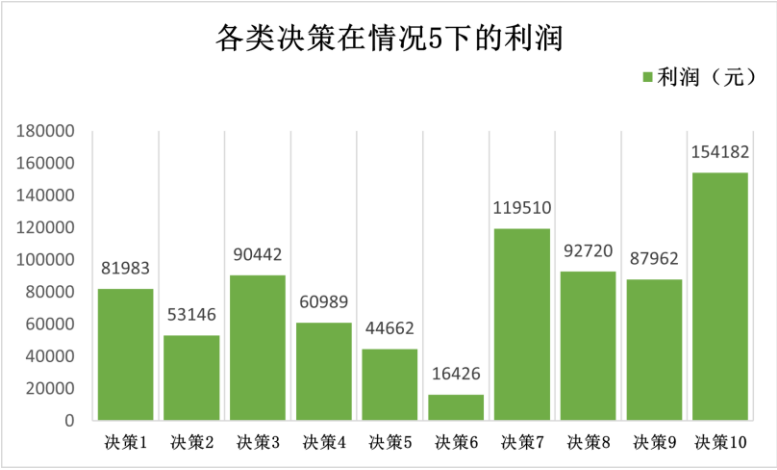


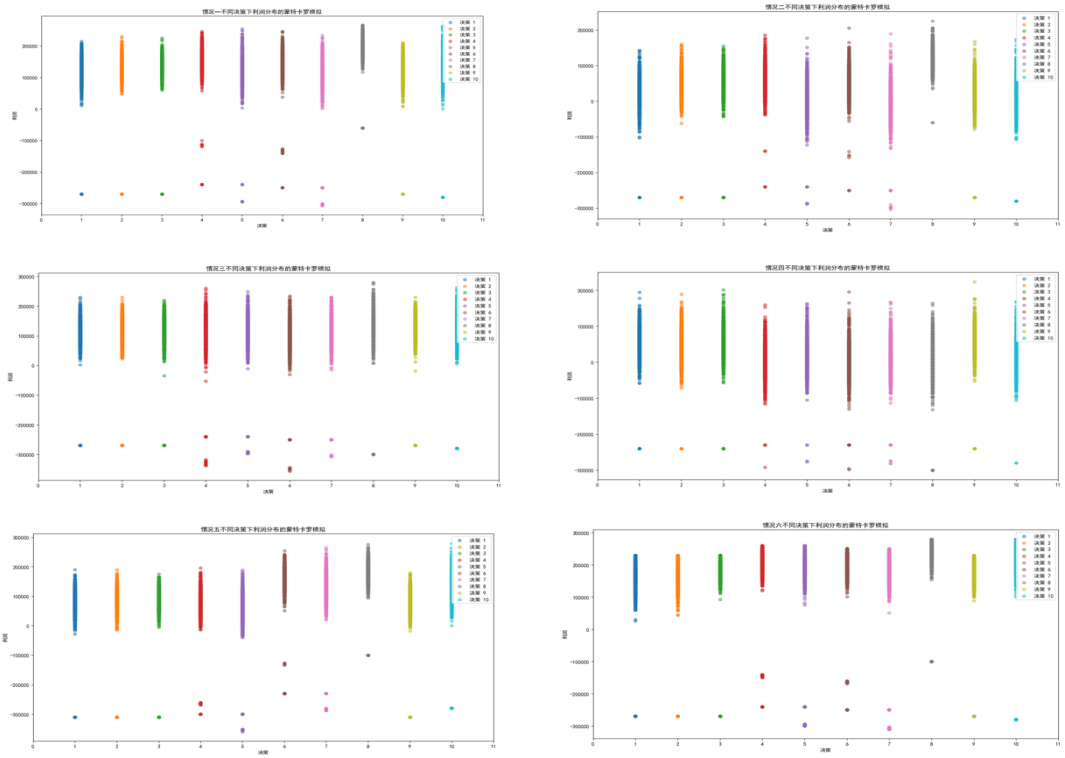
表 14：企业生产中的第六种情况

决策方案	零件一检测	零件二检测	成品检测	拆解	利润（元）
决策 1	1	1	1	1	127024
决策 2	1	1	1	0	74964
决策 3	1	1	0	1	123520

决策 4	1	1	0	0	71980
决策 5	1	0	0	0	9604
决策 6	1	0	1	0	12348
决策 7	0	1	0	0	9123
决策 8	0	1	1	0	11926
决策 9	0	0	1	0	7400
决策 10	0	0	0	0	7400



下图为蒙特卡洛假设的六种情况的可视化：



5.1 问题四对问题三的重新解答

5.1.1 问题背景

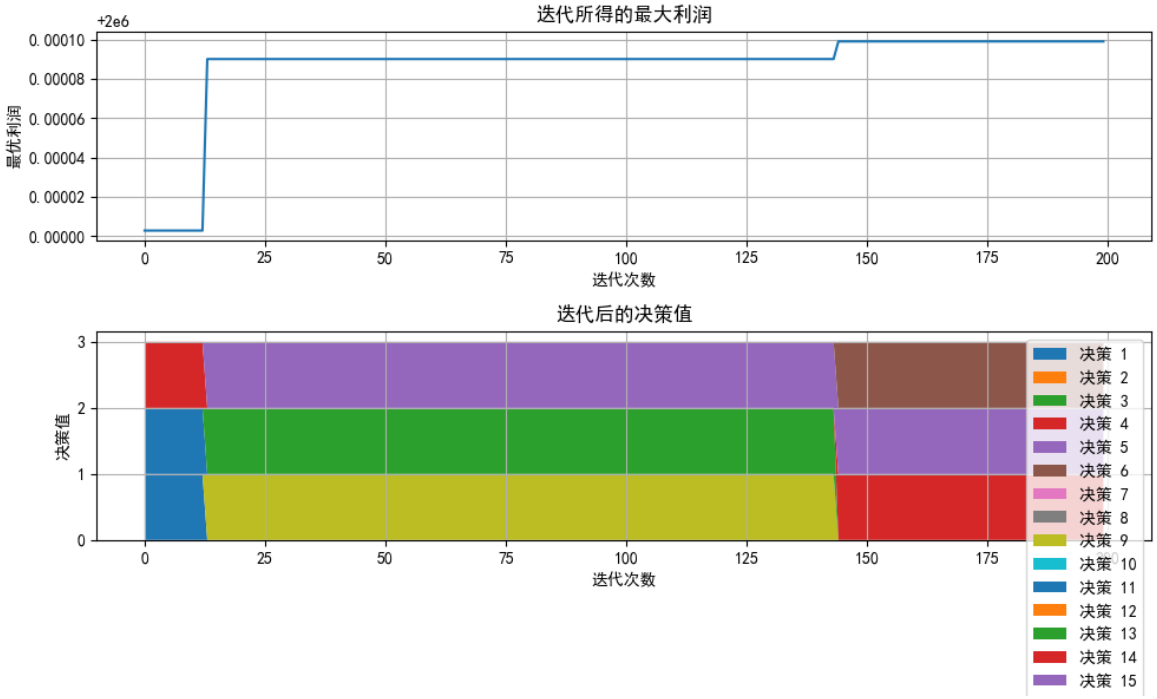
常规做法是同样生成 10000 组零件数据，并且使用 10%标称度设定的 $p0$、$p1$,对这些数据进行序贯检验得出 10000 组新次品率并取平均值。考虑到数据量较大，随机

数据造成的差距可忽略不记，且操作方法、数据参数与上一问完全相同，因此为了节省重复的计算以提高企业决策的效率，计算结果可视作与上一问在 10%标称度下对应的新次品率相同。

沿用样本次品率后，继续使用遗传算法，步骤及思想同第三问一致，故不再过多赘述。

5.1.2 模型的求解

结果由下图所示：



将题目中所给数据输入程序并计算，得到结果如图所示，执行决策 14、15、16 为最优决策方案。

决策 1-8 分别为对 8 个零配件进行检测

决策 9-11 分别为对 3 个半成品进行检测

决策 12 为对成品检测

决策 13-15 为是否对 3 个半成品进行拆解

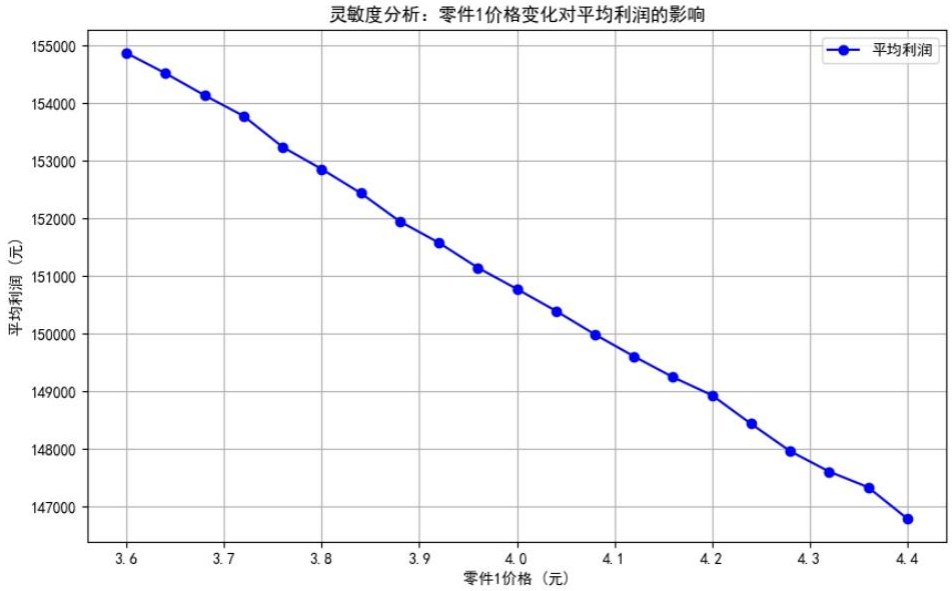
决策 16 为是否拆解成品

六、 模型的分析与检验

6.1 灵敏度分析

在问题二中，零件 1 或者零件 2 的购买单价会对企业最终获得的利润产生巨大的影响。因此，我们蒙特卡落模拟中的利润函数的零件 1 的购买单价进行灵敏度检验，使其值在 10%上下波动，步长为 1%，并且绘制出随着零件 1 购买单价变化，总体利

润变化的情况。这里选择的是问题二的情况 1 中的决策三进行绘制。如下图所示：



从表可看出平均利润和零件价格呈现明显的线性关系，比较稳定。由于利润函数也是线性函数，可以验证蒙特卡洛模拟具有较好的灵敏性。

6.2 误差分析

1. 第一问中，我们采用序贯检验来抽样，在早期的样本量不足，检测可能会提前结束，从而导致选择错误的决策（比如过早接受或拒绝假设）。
2. 第二问中，由于蒙特卡洛模拟基于随机抽样，因此存在一定的随机性误差。即便输入参数完全准确，不同的模拟运行可能会得到不同的结果。
3. 第三问中，进行轮盘赌选择法时，初始种群的选择具有随机性，，在多次迭代后，种群中的不同个体可能逐渐趋于同质化，导致选择范围受限，难以突破局部最优。

七、 模型的评价、改进与推广

7.1 模型的优点

1. 采用序贯检验和假设检验相结合的模型。假设检验可以通过利用已抽取的样本量为抽样设定下界，保证抽样数达到某一数值时可以做出决策，且这一数值可以通过规定允许误差的大小直接求得。而序贯检验逐步抽样，实时更新检测结果的特点实现了只少量抽测样本便能判断假设的可能性，大幅拔高了模型的上限。为企业抽样检测次品率提供了一种既保险又上限高的抽样方案。
2. 蒙特卡洛模拟，灵活度高，能够模拟各类方案的实现，在处理高维问题时能够很好适应多个因素的相互影响，可通过增加模拟的次数来提高精度。
3. 遗传算法，具有强大的多目标优化能力和鲁棒性，对初始条件和参数设置要求不苛刻，同时能够进行全局搜索，更利于找到全局最优解。

7.2 模型的缺点

1. 蒙特卡洛模拟收敛速度慢，依赖于大量随机样本，收敛到高精度解需要大量计算。
2. 遗传算法可能陷入局部最优，尽管遗传算法有全局搜索能力，但在一些复杂问题中，仍有可能因为过早收敛或种群多样性丧失而陷入局部最优。

7.3 模型的推广

序贯检验和假设检验相结合的模型推广

1. 尝试适应性序贯检验：根据实时数据动态调整检验规则和阈值，提高灵活性和准确性。
2. 将模型推广到不同领域的问题中，如医疗领域的疾病筛查、金融领域的风险评估等。根据特定领域的需求调整模型的参数和方法。

八、 参考文献

[1] 冉慧,杜泽弘,王龙飞,等. 基于假设检验的天基仅测角目标定位方法 [J]. 上海航天(中英文), 2024, 41 (04): 155-162.

[2] 李拂晓,肖燕婷,陈占寿. 累积 Logistic 回归模型结构变点的序贯检验 [J]. 应用数学, 2022, 35 (02): 402-411.

[3] 吴勇,王晓洋. 基于蒙特卡罗模拟的汽车空调故障树分析 [J]. 汽车实用技术, 2024, 49 (12): 101-106.

[4] 彭静,罗灿,马佳,等. 基于遗传算法的身管-弹丸接触碰撞网络模型优化研究 [J/OL]. 力学学报, 1-14[2024-09-08].

[5] 盛骤, 谭浩强, 潘承毅. 概率论与数理统计(第五版). 浙江大学出版社, 2019.

[6] 司守奎, 王林. 数学建模算法与应用. 国防工业出版社, 2021：182-222.

[7] 姜启源, 谢金星, 叶俊. 数学模型(第五版). 高等教育出版社, 2018.

附录

附录 1

介绍：支撑材料的文件列表

1. **图片**
2. **数据**
3. **代码**



名称	修改日期	类型	大小
第三问.py	2024-09-08 8:08	Python File	7 KB
假设检验.py	2024-09-08 8:16	Python File	1 KB
问题二.cpp	2024-09-07 20:36	C++ Source File	3 KB
问题四的问题二.cpp	2024-09-08 10:20	C++ Source File	4 KB
问题四的问题三.py	2024-09-08 8:09	Python File	7 KB
序贯检验.cpp	2024-09-08 8:16	C++ Source File	1 KB

附录 2

该代码由 Python 语言编写，用于实现假设检验解决第一问

```python
z_alpha = 1.28
p = 0.1
E = 0.05
n = z_alpha**2 * p * (1 - p) / E**2
    print(n)
```

附录 3

该代码由 C++语言编写，用于实现序贯检验解决第一问

```cpp
#include <bits/stdc++.h>
using namespace std;

const int N = 6;

int stop_test(double p0, double p1, int* data, int n, double
upper_bound=A, double lower_bound=B) {
    double llr = 0;
    for (int i = 0; i < n; i++) {
        if (data[i])
         llr += log((1 - p1) / (1 - p0));
        else
         llr += log(p1 / p0);
        if (llr > upper_bound)
```

```
        return -1;
                if (llr < lower_bound)
                        return i;
    }
    return 0;
}

int main()
{
        double a = 0.1;
    double b = 0.1;
    A = log((1 - b) / a), B = log(b / (1 - a));
        int data[] = {0, 0, 1, 1, 1, 0};
        p0 = 0.1;
        p1 = 0.15;
        int stop = stop_test(p0, p1, data, 6)
        cout << stop_test << endl;
        return 0;
    }
```

附录 4

该代码由 C++代码编写，用于实现蒙特卡罗模拟解决第二问

```
#include <bits/stdc++.h>
using namespace std;
typedef double db;

const int N = 10005, M = 10000;
const int n = 10000; // 采购两种零件各多少个
const int wa = 4, wb = 18, wc = 6; // 购买单价 + 装配成本
const int ca = 2, cb = 3, cc = 3; // 检测成本
const int val = 56, change = 10, des = 40; // 售价 调换损失 拆解
const int d[5] = {0, 0, 0, 1, 0}; // 决策变量
db pa = 0.05, pb = 0.05, pc = 0.05; // 次品率
db cost, profit;
int t = 1, ans = 0, price;
queue<int> ga, gb, na, nb;

void init()
{
        while (ga.size()) ga.pop();
        while (gb.size()) gb.pop();
        while (na.size()) na.pop();
        while (nb.size()) nb.pop();
```

```
        cost = profit = 0;
}

void solve() {
        price = wa + wb + wc + d[1] * ca + d[2] * cb + d[3] * cc; // 一个用于替换的新品的成本
        init();
        int a[N], b[N];
        random_device rd;
        mt19937 mt(rd());
        for (int i = 1; i <= M; i++) { // 生成零件数据
                int rnd = mt() % 100 + 1;
                if (rnd <= pa * 100)
                        a[i] = 0;
                else
                        a[i] = 1;
                rnd = mt() % 100 + 1;
                if (rnd <= pb * 100)
                        b[i] = 0;
                else
                        b[i] = 1;
        }
        cost = M * (wa + wb + d[1] * ca + d[2] * cb);
        for (int i = 1; i <= M; i++) { // 检测零件
                if (a[i] && d[1])
                        ga.push(a[i]);
                else if (!d[1])
                        ga.push(a[i]);
                if (b[i] && d[2])
                        gb.push(b[i]);
                else if (!d[2])
                        gb.push(b[i]);
        }
        while (ga.size() && gb.size()) { // 组装成品
                int ka = ga.front(), kb = gb.front();
                ga.pop();
                gb.pop();
                cost += wc;
                int rnd = mt() % 100 + 1;
                if (!ka || !kb || rnd <= pc * 100) { // 成品不合格
                        if (d[4]) { // 拆解
                                cost += des;
                                if (d[3]) // 检测过则直接回收
                                        cost += cc;
```

```
                        else { // 没检测过则直接售卖然后退换
                                cost += change;
                                profit += val;
                                cost += price;
                        }
                        na.push(ka); // 零件回收
                        nb.push(kb);
                } else { // 丢掉
                        if (d[3])
                                cost += cc;
                        else {
                                profit += val;
                                cost += change;
                                cost += price;
                        }
                }
        } else
                profit += val;
}
int T = 1;
while (T--) { // 回收后重新走相同流程
        while (na.size()) {
                ga.push(na.front());
                na.pop();
        }
        while (nb.size()) {
                gb.push(nb.front());
                nb.pop();
        }
        while (ga.size() && gb.size()) {
                int ka = ga.front(), kb = gb.front();
                ga.pop();
                gb.pop();
                cost += wc;
                int rnd = mt() % 100 + 1;
                if (!ka || !kb || rnd <= pc * 100) {
                        if (d[3])
                                cost += cc;
                        else {
                                profit += val;
                                cost += change;
                                cost += price;
                        }
                } else
```

```
                                profit += val;
                }
        }
        ans += profit - cost;
}

int main() {
        int T = 1000;
        while (T--) solve();
        cout << ans / 1000 << endl;
        return 0;
    }
```

附录 5

该代码由 Python 语言编写，用于实现遗传算法解决第三问

```python
import numpy as np
import random
import matplotlib.pyplot as plt

# 参数设置
gnr = 200     # 迭代次数
pop = 100      # 种群大小
mutation_rate = 0.01      # 变异概率

plt.rcParams['font.sans-serif'] = ['SimHei']  # 用来正常显示中文标签
plt.rcParams['axes.unicode_minus'] = False
# 成本和参数设置
w = np.array([1, 1, 2, 1, 1, 2, 1, 2, 4, 4, 4, 6])  # 检测成本
c = np.array([2, 8, 12, 2, 8, 12, 8, 12, 8, 8, 8, 8])  # 装配成本
product_price = 200  # 产品售价
rc = 40    # 换货损失
disc_parts = 10  # 零件拆解成本
disc_semi = np.array([6, 6, 6])  # 半成品拆解成本

# 次品率 (8 种零件 + 3 种半成品 + 成品)
defect_rates = np.array([0.1] * 12)

# 随机初始化种群
def initialize_population(size, num_vars):
    return np.random.randint(2, size=(size, num_vars))

# 设置每种零件的数量
num_parts = 10000
```

```
# 计算适应度函数，计算总利润
def compute_profit(solution):
    def compute_costs_and_probs(solution, num_parts):
        ttc = 0
        part_def_probs = np.ones(8)
        semi_def_probs = np.ones(3)
        final_def_prob = 1

        # 计算零件阶段的成本和次品率
        for i in range(8):
            if solution[i] == 1:
                ttc += w[i] * num_parts
                part_def_probs[i] *= (1 - defect_rates[i])
            else:
                part_def_probs[i] *= defect_rates[i]
            ttc += c[i] * num_parts

        # 计算半成品阶段的成本和次品率
        for i in range(8, 11):
            semi_idx = i - 8
            if solution[i] == 1:
                ttc += w[i] * num_parts
                semi_def_probs[semi_idx] *= part_def_probs[semi_idx *
3:semi_idx * 3 + 3].prod() * (1 - defect_rates[i])
            else:
                semi_def_probs[semi_idx] *= part_def_probs[semi_idx *
3:semi_idx * 3 + 3].prod() * defect_rates[i]
            ttc += c[i] * num_parts

        # 计算成品阶段的成本和次品率
        if solution[11] == 1:
            ttc += w[11] * num_parts
            final_def_prob *= semi_def_probs.prod() * (1 -
defect_rates[11])
        else:
            final_def_prob *= semi_def_probs.prod() * defect_rates[11]
        ttc += c[11] * num_parts

        return ttc, part_def_probs, semi_def_probs, final_def_prob

    def handle_defective_items(part_def_probs, semi_def_probs,
final_def_prob, num_parts):
        ttc = 0
```

```
        total_profit = 0

        # 计算收入和预期损失
        rev = product_price * num_parts
        expected_loss = final_def_prob * (rc + product_price) * num_parts
        total_profit = rev - ttc - expected_loss

        # 处理拆解逻辑
        for i in range(3):
            if solution[12 + i] == 1 and semi_def_probs[i] > 0:
                ttc += disc_semi[i] * num_parts
                sub_cost, new_part_probs, _, _ =
compute_costs_and_probs(np.concatenate([solution[:8], [0, 0, 0, 0]]),
num_parts)
                ttc += sub_cost
                part_def_probs = new_part_probs

        if solution[15] == 1 and final_def_prob > 0:
            ttc += disc_parts * num_parts
            sub_cost, new_part_probs, _, _ =
compute_costs_and_probs(np.concatenate([solution[:8], [0, 0, 0, 0]]),
num_parts)
            ttc += sub_cost
            part_def_probs = new_part_probs

        return ttc, total_profit

    # 计算生产成本和次品率
    ttc, part_def_probs, semi_def_probs, final_def_prob =
compute_costs_and_probs(solution, num_parts)

    # 处理拆解后的零件
    ttc, total_profit = handle_defective_items(part_def_probs,
semi_def_probs, final_def_prob, num_parts)

    return total_profit

# 选择函数（轮盘赌选择）
def select_individuals(population, fitness):
    total_fitness = sum(fitness)
    probabilities = [f / total_fitness for f in fitness]
    selected_index = np.random.choice(len(population), size=pop,
p=probabilities)
    return population[selected_index]
```

```
# 交叉函数（单点交叉）
def crossover(parent1, parent2):
    crossover_point = random.randint(0, len(parent1) - 1)
    child1 = np.concatenate([parent1[:crossover_point],
parent2[crossover_point:]])
    child2 = np.concatenate([parent2[:crossover_point],
parent1[crossover_point:]])
    return child1, child2

# 变异函数
def mutate(individual, mutation_rate):
    for i in range(len(individual)):
        if random.random() < mutation_rate:
            individual[i] = 1 - individual[i]
    return individual

# 遗传算法主循环
def genetic_algorithm():
    population = initialize_population(pop, 16)  # 16 个决策变量
    best_solution = None
    best_profit = -float('inf')
    best_profit_history = []
    decision_history = []

    for generation in range(gnr):
        fitness = [compute_profit(individual) for individual in population]

        # 记录最佳解
        current_best_profit = max(fitness)
        if current_best_profit > best_profit:
            best_profit = current_best_profit
            best_solution = population[np.argmax(fitness)]

        best_profit_history.append(best_profit)
        decision_history.append(best_solution.copy())  # 记录当前代的最优解决
策

        # 选择个体
        selected_population = select_individuals(population, fitness)

        # 生成下一代
        next_generation = []
        for i in range(0, pop, 2):
```

```
            parent1, parent2 = selected_population[i],
selected_population[i+1]
            child1, child2 = crossover(parent1, parent2)
            next_generation.append(mutate(child1, mutation_rate))
            next_generation.append(mutate(child2, mutation_rate))

        population = np.array(next_generation)

        print(f"Generation {generation + 1}: Best Profit = {best_profit}")

    # 绘制最优解波动图
    plt.figure(figsize=(10, 6))

    # 绘制最佳利润波动图
    plt.subplot(2, 1, 1)
    plt.plot(range(gnr), best_profit_history)
    plt.xlabel('迭代次数')
    plt.ylabel('最优利润')
    plt.title('迭代所得的最大利润')
    plt.grid(True)

    # 绘制决策变量的变化情况（堆叠面积图）
    decision_history = np.array(decision_history)
    plt.subplot(2, 1, 2)
    plt.stackplot(range(gnr), decision_history.T, labels=[f'决策 {i+1}' for
i in range(decision_history.shape[1])])
    plt.xlabel('迭代次数')
    plt.ylabel('决策值')
    plt.title('迭代后的决策值')
    plt.legend(loc='upper right')
    plt.grid(True)

    plt.tight_layout()
    plt.show()

    return best_solution, best_profit

# 运行遗传算法
best_solution, best_profit = genetic_algorithm()

print(f"Best Solution: {best_solution}")
    print(f"Best Profit: {best_profit} 元")
```

附录 6

该代码是由 C++代码编写，用于实现蒙特卡罗模拟解决第 4 问中对问题 2 的重新解答

```
#include <bits/stdc++.h>
using namespace std;
typedef double db;

const int N = 10005, M = 10000;
const int n = 10000; // 采购两种零件各多少个
const int wa = 4, wb = 18, wc = 6; // 购买单价 + 装配成本
const int ca = 8, cb = 1, cc = 2; // 检测成本
const int val = 56, change = 10, des = 5; // 售价 调换损失 拆解
const int d[15][5] = {0, 1, 1, 1, 1,
                                          0, 1, 1, 0, 1,
                                          0, 1, 1, 0, 0,
                                          0, 1, 0, 0, 0,
                                          0, 1, 0, 1, 0,
                                          0, 0, 1, 0, 0,
                                          0, 0, 1, 1, 0,
                                          0, 0, 0, 0, 0,
                                          0, 1, 1, 1, 0,
                                          0, 0, 0, 1, 0
                                         }; // 决策变量
db PA = 0.1, PB = 0.2, pc = 0.1, pa, pb; // 次品率
db cost, profit;
int t = 1, ans, price, a[N], b[N];
queue<int> ga, gb, na, nb;

void init()
{
        while (ga.size()) ga.pop();
        while (gb.size()) gb.pop();
        while (na.size()) na.pop();
        while (nb.size()) nb.pop();
        cost = profit = 0;
}

int stop_test(double p0, double p1, int* data, int n, double
upper_bound=2.89, double lower_bound=-2.25) {
    double llr = 0;
    for (int i = 1; i <= M; i++) {
        if (data[i])
         llr += log((1 - p1) / (1 - p0));
        else
         llr += log(p1 / p0);
```

```
        if (llr > upper_bound)
        return -1;
                if (llr < lower_bound)
                        return i;
    }
    return 0;
}

void checka()
{
        double p0 = PA;
    double p1 = PA + 0.05;
    random_device rd;
    mt19937 mt(rd());
        for (int i = 1; i <= M; i++) {
                int rnd = mt() % 100 + 1;
                if (rnd <= PA * 100)
                        a[i] = 0;
                else
                        a[i] = 1;
        }
        double res = stop_test(p0, p1, a, n);
        double sum = 0;
        for (int i = 1; i <= res; i++)
                sum += a[i];
        pa = 1 - sum / res;
}

void checkb()
{
        double p0 = PB;
    double p1 = PB + 0.05;
    random_device rd;
    mt19937 mt(rd());
        for (int i = 1; i <= M; i++) {
                int rnd = mt() % 100 + 1;
                if (rnd <= PB * 100)
                        b[i] = 0;
                else
                        b[i] = 1;
        }
        double res = stop_test(p0, p1, b, n);
        double sum = 0;
        for (int i = 1; i <= res; i++)
```

```
                sum += b[i];
        pb = 1 - sum / res;
}

void solve(int kkk) {
        random_device rd;
    mt19937 mt(rd());
        price = wa + wb + wc + d[kkk][1] * ca + d[kkk][2] * cb + d[kkk][3]
* cc;
        init();
        checka(); checkb();
        cost = M * (wa + wb + d[kkk][1] * ca + d[kkk][2] * cb);
        pc = pa;
        for (int i = 1; i <= M; i++) {
                int rnd = mt() % 10000 + 1;
                if (rnd <= pa * 10000)
                        a[i] = 0;
                else
                        a[i] = 1;
        }
        for (int i = 1; i <= M; i++) {
                int rnd = mt() % 10000 + 1;
                if (rnd <= pb * 10000)
                        b[i] = 0;
                else
                        b[i] = 1;
        }
        for (int i = 1; i <= M; i++) {
                if (a[i] && d[kkk][1])
                        ga.push(a[i]);
                else if (!d[kkk][1])
                        ga.push(a[i]);
                if (b[i] && d[kkk][2])
                        gb.push(b[i]);
                else if (!d[kkk][2])
                        gb.push(b[i]);
        }
        while (ga.size() && gb.size()) {
                int ka = ga.front(), kb = gb.front();
                ga.pop();
                gb.pop();
                cost += wc;
                int rnd = mt() % 100 + 1;
                if (!ka || !kb || rnd <= pc * 100) { // 成品不合格
```

```
                        if (d[kkk][4] && d[kkk][1] && d[kkk][2]) { // 拆解
                                cost += des;
                                if (d[kkk][3])
                                        cost += cc;
                                else {
                                        cost += change;
                                        profit += val;
                                        cost += price;
                                }
                                na.push(ka);
                                nb.push(kb);
                        } else { // 丢掉
                                if (d[kkk][3])
                                        cost += cc;
                                else {
                                        profit += val;
                                        cost += change;
                                        cost += price;
                                }
                        }
                } else
                        profit += val;
        }
        int T = 1;
        while (T--) { // 回收
                while (na.size()) {
                        ga.push(na.front());
                        na.pop();
                }
                while (nb.size()) {
                        gb.push(nb.front());
                        nb.pop();
                }
                while (ga.size() && gb.size()) {
                        int ka = ga.front(), kb = gb.front();
                        ga.pop();
                        gb.pop();
                        cost += wc;
                        int rnd = mt() % 100 + 1;
                        if (!ka || !kb || rnd <= pc * 100) {
                                if (d[kkk][3])
                                        cost += cc;
                                else {
                                        profit += val;
```

```
                                        cost += change;
                                        cost += price;
                                }
                        } else
                                profit += val;
                }
        }
        ans += profit - cost;
}

int main() {
        for (int i = 0; i < 10; i++) {
                int T = 1000;
                ans = 0;
                while (T--) solve(i);
                cout << ans / 1000 << endl;
        }
        return 0;
    }
```

附录 7

该代码由 Python 语言编写，用于实现遗传算法解决第 4 问对问题 3 的重新解答

```
import numpy as np
import random
import matplotlib.pyplot as plt

# 参数设置
gnr = 200     # 迭代次数
pop = 150      # 种群大小
mutation_rate = 0.01      # 变异概率

plt.rcParams['font.sans-serif'] = ['SimHei']  # 用来正常显示中文标签
plt.rcParams['axes.unicode_minus'] = False
# 成本和参数设置
w = np.array([1, 1, 2, 1, 1, 2, 1, 2, 4, 4, 4, 6])  # 检测成本
c = np.array([2, 8, 12, 2, 8, 12, 8, 12, 8, 8, 8, 8])  # 装配成本
product_price = 200  # 产品售价
rc = 40    # 换货损失
disc_parts = 10  # 零件拆解成本
disc_semi = np.array([6, 6, 6])  # 半成品拆解成本

# 次品率 (8 种零件 + 3 种半成品 + 成品)
defect_rates = np.array([0.0931677] * 12)
```

```
# 随机初始化种群
def initialize_population(size, num_vars):
    return np.random.randint(2, size=(size, num_vars))

# 设置每种零件的数量
num_parts = 10000

# 计算适应度函数，计算总利润
def compute_profit(solution):
    def compute_costs_and_probs(solution, num_parts):
        ttc = 0
        part_def_probs = np.ones(8)
        semi_def_probs = np.ones(3)
        final_def_prob = 1

        # 计算零件阶段的成本和次品率
        for i in range(8):
            if solution[i] == 1:
                ttc += w[i] * num_parts
                part_def_probs[i] *= (1 - defect_rates[i])
            else:
                part_def_probs[i] *= defect_rates[i]
            ttc += c[i] * num_parts

        # 计算半成品阶段的成本和次品率
        for i in range(8, 11):
            semi_idx = i - 8
            if solution[i] == 1:
                ttc += w[i] * num_parts
                semi_def_probs[semi_idx] *= part_def_probs[semi_idx *
3:semi_idx * 3 + 3].prod() * (1 - defect_rates[i])
            else:
                semi_def_probs[semi_idx] *= part_def_probs[semi_idx *
3:semi_idx * 3 + 3].prod() * defect_rates[i]
            ttc += c[i] * num_parts

        # 计算成品阶段的成本和次品率
        if solution[11] == 1:
            ttc += w[11] * num_parts
            final_def_prob *= semi_def_probs.prod() * (1 -
defect_rates[11])
        else:
            final_def_prob *= semi_def_probs.prod() * defect_rates[11]
```

```
        ttc += c[11] * num_parts

        return ttc, part_def_probs, semi_def_probs, final_def_prob

    def handle_defective_items(part_def_probs, semi_def_probs,
final_def_prob, num_parts):
        ttc = 0
        total_profit = 0

        # 计算收入和预期损失
        rev = product_price * num_parts
        expected_loss = final_def_prob * (rc + product_price) * num_parts
        total_profit = rev - ttc - expected_loss

        # 处理拆解逻辑
        for i in range(3):
            if solution[12 + i] == 1 and semi_def_probs[i] > 0:
                ttc += disc_semi[i] * num_parts
                sub_cost, new_part_probs, _, _ =
compute_costs_and_probs(np.concatenate([solution[:8], [0, 0, 0, 0]]),
num_parts)
                ttc += sub_cost
                part_def_probs = new_part_probs

        if solution[15] == 1 and final_def_prob > 0:
            ttc += disc_parts * num_parts
            sub_cost, new_part_probs, _, _ =
compute_costs_and_probs(np.concatenate([solution[:8], [0, 0, 0, 0]]),
num_parts)
            ttc += sub_cost
            part_def_probs = new_part_probs

        return ttc, total_profit

    # 计算生产成本和次品率
    ttc, part_def_probs, semi_def_probs, final_def_prob =
compute_costs_and_probs(solution, num_parts)

    # 处理拆解后的零件
    ttc, total_profit = handle_defective_items(part_def_probs,
semi_def_probs, final_def_prob, num_parts)

    return total_profit
```

```
# 选择函数（轮盘赌选择）
def select_individuals(population, fitness):
    total_fitness = sum(fitness)
    probabilities = [f / total_fitness for f in fitness]
    selected_index = np.random.choice(len(population), size=pop,
p=probabilities)
    return population[selected_index]

# 交叉函数（单点交叉）
def crossover(parent1, parent2):
    crossover_point = random.randint(0, len(parent1) - 1)
    child1 = np.concatenate([parent1[:crossover_point],
parent2[crossover_point:]])
    child2 = np.concatenate([parent2[:crossover_point],
parent1[crossover_point:]])
    return child1, child2

# 变异函数
def mutate(individual, mutation_rate):
    for i in range(len(individual)):
        if random.random() < mutation_rate:
            individual[i] = 1 - individual[i]
    return individual

# 遗传算法主循环
def genetic_algorithm():
    population = initialize_population(pop, 16)  # 16 个决策变量
    best_solution = None
    best_profit = -float('inf')
    best_profit_history = []
    decision_history = []

    for generation in range(gnr):
        fitness = [compute_profit(individual) for individual in population]

        # 记录最佳解
        current_best_profit = max(fitness)
        if current_best_profit > best_profit:
            best_profit = current_best_profit
            best_solution = population[np.argmax(fitness)]

        best_profit_history.append(best_profit)
        decision_history.append(best_solution.copy())  # 记录当前代的最优解决
策
```

```
        # 选择个体
        selected_population = select_individuals(population, fitness)

        # 生成下一代
        next_generation = []
        for i in range(0, pop, 2):
            parent1, parent2 = selected_population[i],
selected_population[i+1]
            child1, child2 = crossover(parent1, parent2)
            next_generation.append(mutate(child1, mutation_rate))
            next_generation.append(mutate(child2, mutation_rate))

        population = np.array(next_generation)

        print(f"Generation {generation + 1}: Best Profit = {best_profit}")

    # 绘制最优解波动图
    plt.figure(figsize=(10, 6))

    # 绘制最佳利润波动图
    plt.subplot(2, 1, 1)
    plt.plot(range(gnr), best_profit_history)
    plt.xlabel('迭代次数')
    plt.ylabel('最优利润')
    plt.title('迭代所得的最大利润')
    plt.grid(True)

    # 绘制决策变量的变化情况（堆叠面积图）
    decision_history = np.array(decision_history)
    plt.subplot(2, 1, 2)
    plt.stackplot(range(gnr), decision_history.T, labels=[f'决策 {i+1}' for
i in range(decision_history.shape[1])])
    plt.xlabel('迭代次数')
    plt.ylabel('决策值')
    plt.title('迭代后的决策值')
    plt.legend(loc='upper right')
    plt.grid(True)

    plt.tight_layout()
    plt.show()

    return best_solution, best_profit
```

```
# 运行遗传算法
best_solution, best_profit = genetic_algorithm()

print(f"Best Solution: {best_solution}")
    print(f"Best Profit: {best_profit} 元")
```